

CSE 3204
Formal Language, Automata and Computability



Lecture 18
Closure Properties of CFL

Substitution

- Σ is an alphabet and for each $a \in \Sigma$ there is a language L_a .
- s is a substitution on Σ such that $s(a) = L_a$ for each a .

If $w = a_1 a_2 \cdots a_n$ is a string in Σ^* , then $s(w)$ is the language of all strings $x_1 x_2 \cdots x_n$ such that string x_i is in the language $s(a_i)$, for $i = 1, 2, \dots, n$. Put another way, $s(w)$ is the concatenation of the languages $s(a_1) s(a_2) \cdots s(a_n)$.

Example 7.22: Suppose $s(0) = \{a^n b^n \mid n \geq 1\}$ and $s(1) = \{aa, bb\}$. That is, s is a substitution on alphabet $\Sigma = \{0, 1\}$. Language $s(0)$ is the set of strings

Let $w = 01$. Then $s(w)$ is the concatenation of the languages $s(0)s(1)$. To be exact, $s(w)$ consists of all strings of the forms $a^n b^n aa$ and $a^n b^{n+2}$, where $n \geq 1$.

Substitution

Now, suppose $L = L(0^*)$, that is, the set of all strings of 0's. Then $s(L) = (s(0))^*$. This language is the set of all strings of the form

$$a^{n_1} b^{n_1} a^{n_2} b^{n_2} \dots a^{n_k} b^{n_k}$$

for some $k \geq 0$ and any sequence of choices of positive integers $n_1, n_2, \dots, n_k$. It includes strings such as ϵ , $aabbaaabb$, and $abaabbabab$. $\square$

Theorem 7.23: If L is a context-free language over alphabet Σ , and s is a substitution on Σ such that $s(a)$ is a CFL for each a in Σ , then $s(L)$ is a CFL.

Homomorphism

- A mapping that replaces each symbol of an alphabet with a string over another alphabet.

Σ =input alphabet Γ =output alphabet

- A homomorphism h is a function $h: \Sigma \rightarrow \Gamma^*$ that maps each symbol in Σ to a string (may be empty) over Γ

Applications of the Substitution Theorem

Theorem 7.24: The context-free languages are closed under the following operations:

1. Union.
2. Concatenation.
3. Closure (*), and positive closure (+).
4. Homomorphism.

PROOF: Each requires only that we set up the proper substitution. The proofs below each involve substitution of context-free languages into other context-free languages, and therefore produce CFL's by Theorem 7.23.

1. *Union:* Let L_1 and L_2 be CFL's. Then $L_1 \cup L_2$ is the language $s(L)$, where L is the language $\{1, 2\}$, and s is the substitution defined by $s(1) = L_1$ and $s(2) = L_2$.
2. *Concatenation:* Again let L_1 and L_2 be CFL's. Then $L_1 L_2$ is the language $s(L)$, where L is the language $\{12\}$, and s is the same substitution as in case (1).

3. *Closure and positive closure:* If L_1 is a CFL, L is the language $\{1\}^*$, and s is the substitution $s(1) = L_1$, then $L_1^* = s(L)$. Similarly, if L is instead the language $\{1\}^+$, then $L_1^+ = s(L)$.

4. Suppose L is a CFL over alphabet Σ , and h is a homomorphism on Σ . Let s be the substitution that replaces each symbol a in Σ by the language consisting of the one string that is $h(a)$. That is, $s(a) = \{h(a)\}$, for all a in Σ . Then $h(L) = s(L)$.

CFL is closed under reversal.

Theorem 7.25: If L is a CFL, then so is L^R .

Intersection with a Regular Language

- CFL is not closed under intersection.

$$L = \{0^n 1^n 2^n \mid n \geq 1\}$$

is not a context-free language. However, the following two languages *are* context-free:

$$L_1 = \{0^n 1^n 2^i \mid n \geq 1, i \geq 1\}$$

$$L_2 = \{0^i 1^n 2^n \mid n \geq 1, i \geq 1\}$$

A grammar for L_1 is:

$$\begin{aligned} S &\rightarrow AB \\ A &\rightarrow 0A1 \mid 01 \\ B &\rightarrow 2B \mid 2 \end{aligned}$$

In this grammar, A generates all strings of the form $0^n 1^n$, and B generates all strings of 2's. A grammar for L_2 is:

$$\begin{aligned} S &\rightarrow AB \\ A &\rightarrow 0A \mid 0 \\ B &\rightarrow 1B2 \mid 12 \end{aligned}$$

However, $L = L_1 \cap L_2$. To see why, observe that L_1 requires that there be the same number of 0's and 1's, while L_2 requires the numbers of 1's and 2's to be equal. A string in both languages must have equal numbers of all three symbols and thus be in L .

If the CFL's were closed under intersection, then we could prove the false statement that L is context-free. We conclude by contradiction that the CFL's are not closed under intersection. $\square$

Intersection with a Regular Language

Theorem 7.27 : If L is a CFL and R is a regular language, then $L \cap R$ is a CFL.

Formally, let

$$P = (Q_P, \Sigma, \Gamma, \delta_P, q_P, Z_0, F_P)$$

be a PDA that accepts L by final state, and let

$$A = (Q_A, \Sigma, \delta_A, q_A, F_A)$$

be a DFA for R . Construct PDA

$$P' = (Q_P \times Q_A, \Sigma, \Gamma, \delta, (q_P, q_A), Z_0, F_P \times F_A)$$

where $\delta((q, p), a, X)$ is defined to be the set of all pairs $((r, s), \gamma)$ such that:

1. $s = \hat{\delta}_A(p, a)$, and
2. Pair (r, γ) is in $\delta_P(q, a, X)$.

It is an easy induction on the numbers of moves made by the PDA's that $(q_P, w, Z_0) \xrightarrow{*}_D (q, \epsilon, \gamma)$ if and only if $((q_P, q_A), w, Z_0) \xrightarrow{*}_{D'} ((q, p), \epsilon, \gamma)$, where

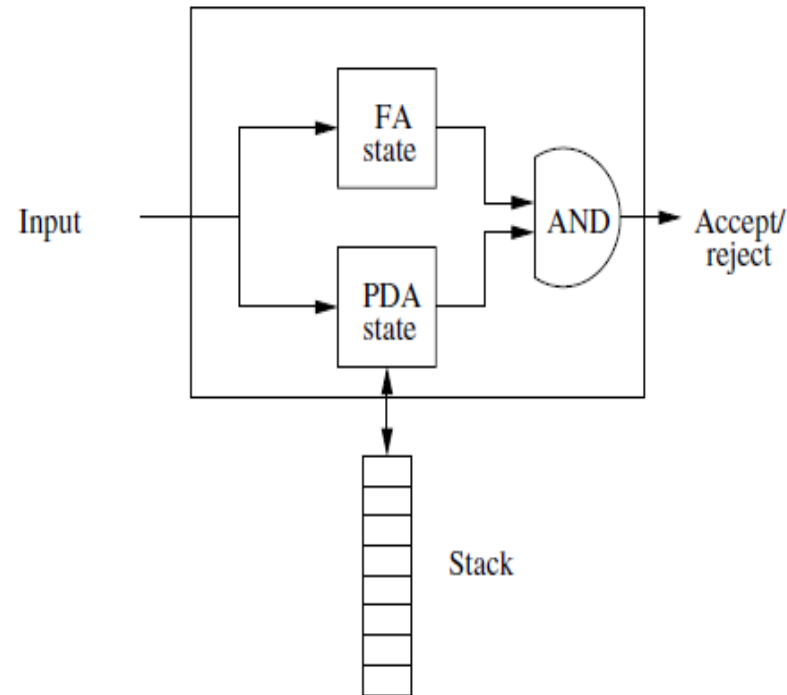


Figure 7.9: A PDA and a FA can run in parallel to create a new PDA

Intersection with a Regular Language--Example

L. The PDA F was defined by

$$P_F = (\{p, q, r\}, \{i, e\}, \{Z, X_0\}, \delta_F, p, X_0, \{r\})$$

where δ_F consists of the rules:

1. $\delta_F(p, \epsilon, X_0) = \{(q, ZX_0)\}$.
2. $\delta_F(q, i, Z) = \{(q, ZZ)\}$.
3. $\delta_F(q, e, Z) = \{(q, \epsilon)\}$.
4. $\delta_F(q, \epsilon, X_0) = \{(r, \epsilon)\}$.

Now, let us introduce a finite automaton

$$A = (\{s, t\}, \{i, e\}, \delta_A, s, \{s, t\})$$

that accepts the strings in the language of $\mathbf{i^*e^*}$, that is, all strings of i 's followed by e 's. Call this language R . Transition function δ_A is given by the rules:

- a) $\delta_A(s, i) = s$.
- b) $\delta_A(s, e) = t$.
- c) $\delta_A(t, e) = t$.

We shall construct a PDA

$$P = (\{p, q, r\} \times \{s, t\}, \{i, e\}, \{Z, X_0\}, \delta, (p, s), X_0, \{r\} \times \{s, t\})$$

- 1: $\delta((p, s), \epsilon, X_0) = \{((q, s), ZX_0)\}$.
- 2a: $\delta((q, s), i, Z) = \{((q, s), ZZ)\}$.
- 3b: $\delta((q, s), e, Z) = \{((q, t), \epsilon)\}$.
- 4: $\delta((q, s), \epsilon, X_0) = \{((r, s), \epsilon)\}$. Note: one can prove that this rule is never exercised. The reason is that it is impossible to pop the stack without seeing an e , and as soon as P sees an e the second component of its state becomes t .
- 3c: $\delta((q, t), e, Z) = \{((q, t), \epsilon)\}$.
- 4: $\delta((q, t), \epsilon, X_0) = \{((r, t), \epsilon)\}$.

Intersection with a Regular Language--Example

The language $L \cap R$ is the set of strings with some number of i 's followed by one more e , that is, $\{i^n e^{n+1} \mid n \geq 0\}$. This set is exactly those if-else violations that consist of a block of if's followed by a block of else's. The language is evidently a CFL, generated by the grammar with productions $S \rightarrow iSe \mid e$.

Theorem 7.29: The following are true about CFL's L , L_1 , and L_2 , and a regular language R .

1. $L - R$ is a context-free language.
2. $\overline{L}$ is not necessarily a context-free language.
3. $L_1 - L_2$ is not necessarily context-free.

Read by yourself.

Inverse Homomorphism

- Works in the opposite direction of homomorphism.
- Suppose $h : \Sigma^* \rightarrow \Gamma^*$ and a language $L \subseteq \Gamma^*$.

The inverse homomorphism of L under h , denoted by $h^{-1}(L)$, is defined as

$$h^{-1}(L) = \{ w \in \Sigma^* \mid h(w) \in L \}.$$

- Example: $\Sigma = \{a, b\}, \quad \Gamma = \{0, 1\}$

$$h(a) = 0, \quad h(b) = 11, \quad L = \{011\}, \quad h^{-1}(L) = \{ab\}$$

Inverse Homomorphism

Theorem 7.30 : Let L be a CFL and h a homomorphism. Then $h^{-1}(L)$ is a CFL.

PROOF: Suppose h applies to symbols of alphabet Σ and produces strings in T^* . We also assume that L is a language over alphabet T . As suggested above, we start with a PDA $P = (Q, T, \Gamma, \delta, q_0, Z_0, F)$ that accepts L by final state. We construct a new PDA

$$P' = (Q', \Sigma, \Gamma, \delta', (q_0, \epsilon), Z_0, F \times \{\epsilon\}) \quad (7.1)$$

where:

1. Q' is the set of pairs (q, x) such that:
 - (a) q is a state in Q , and
 - (b) x is a suffix (not necessarily proper) of some string $h(a)$ for some input symbol a in Σ .

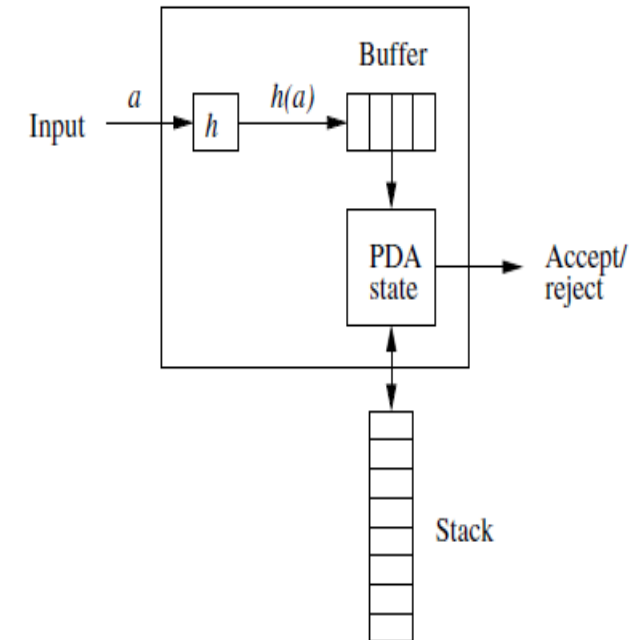


Figure 7.10: Constructing a PDA to accept the inverse homomorphism of what a given PDA accepts

Inverse Homomorphism

That is, the first component of the state of P' is the state of P , and the second component is the buffer. We assume that the buffer will periodically be loaded with a string $h(a)$, and then allowed to shrink from the front, as we use its symbols to feed the simulated PDA P . Note that since Σ is finite, and $h(a)$ is finite for all a , there are only a finite number of states for P'

2. δ' is defined by the following rules:

(a) $\delta'((q, \epsilon), a, X) = \{((q, h(a)), X)\}$ for all symbols a in Σ , all states q in Q , and stack symbols X in Γ . Note that a cannot be ϵ here.

(b) If $\delta(q, b, X)$ contains (p, γ) , where b is in T or $b = \epsilon$, then

$$\delta'((q, bx), \epsilon, X)$$

contains $((p, x), \gamma)$. That is, P' always has the option of simulating a move of P , using the front of its buffer. If b is a symbol in T , then the buffer must not be empty, but if $b = \epsilon$, then the buffer can be empty.

3. Note that, as defined in (7.1), the start state of P' is (q_0, ϵ) ; i.e., P' starts in the start state of P with an empty buffer.
4. Likewise, the accepting states of P' , as per (7.1), are those states (q, ϵ) such that q is an accepting state of P .

The following statement characterizes the relationship between P' and P :

- $(q_0, h(w), Z_0) \stackrel{*}{\vdash}_P (p, \epsilon, \gamma)$ if and only if $((q_0, \epsilon), w, Z_0) \stackrel{*}{\vdash}_{P'} ((p, \epsilon), \epsilon, \gamma)$.

- Section 7.3 of Chapter 7